



§ 6. 指针基础 – 画内存图并分析

要求:

- 1、模仿第06模块PDF课件中 (P. 16-19/P. 27-30) 的样式，画出下列每小题每一步执行的内存分配及指向图示，分析为什么得到最后的结果。
 - ★ PDF课件的P. 31（如何同时得到周长和面积）
 - ★ PDF课件的P. 32（为什么无法进行交换）
 - ★ PDF课件的P. 33（为什么会出现错误，导致错误的关键词是哪一句）
- 2、每个语句要画一张内存状态图，每小题都是4张图
 - ★ 第1张初始内存分配图附件已给出
- 3、不允许手写、手写后贴图
- 4、转换为pdf后在“文档作业”中提交（5.23前）

友情提醒： 本周作业量不大，第2个大作业要在下周指针讲完后才能布置，大家尽快把VS2022调试的文档作业做完(除“字符指针”/“引用”知识点外，其它已经全部讲过了)，千万不要卡DDL!!!



§ 6. 指针基础 – 画内存图并分析

★ PDF课件的P. 31（如何同时得到周长和面积）

```
#include <iostream>
using namespace std;
```

```
#define PI 3.14159
```

```
double SL(double R, double *L)
```

```
{ double S;
```

```
  S = PI*R*R;
```

```
  *L = 2*PI*R;
```

```
  return S;
```

```
}
```

```
int main()
```

```
{ double s, l, r=3;
```

```
  s=SL(r, &l);
```

```
  cout << "s=" << s << endl;
```

```
  cout << "l=" << l << endl;
```

```
}
```

函数执行后同时得到周长及面积

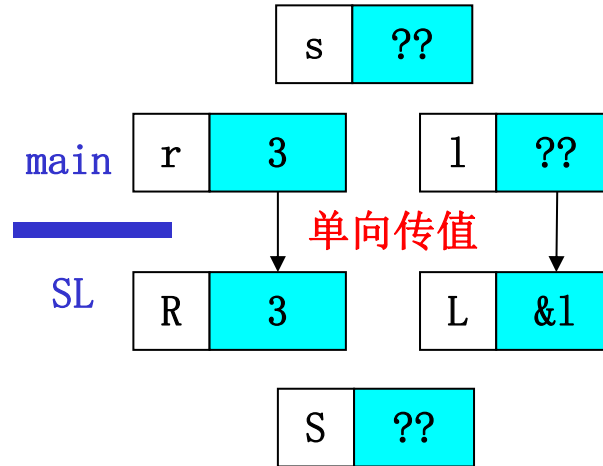
周长：指针变量做形参方式

面积：函数返回值方式

注：函数的return只能带一个返回值！！

s=28. 2743

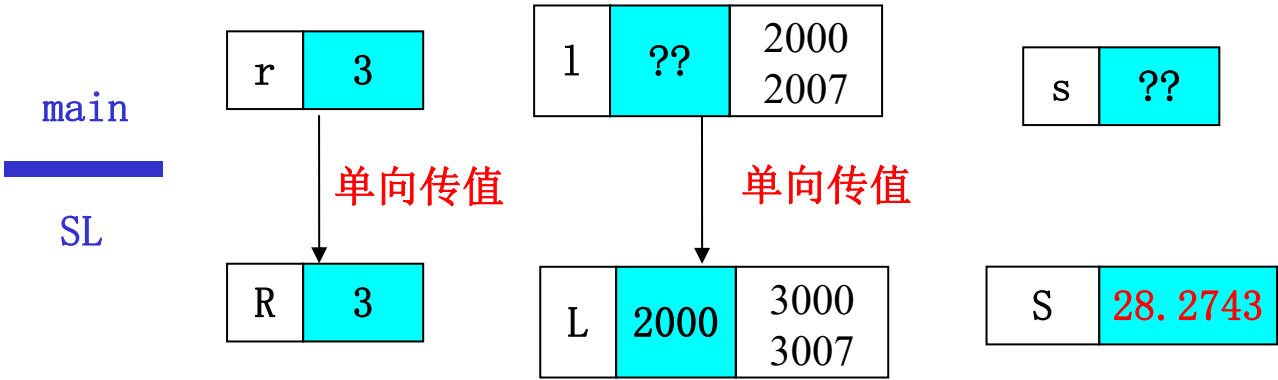
l=18. 8495



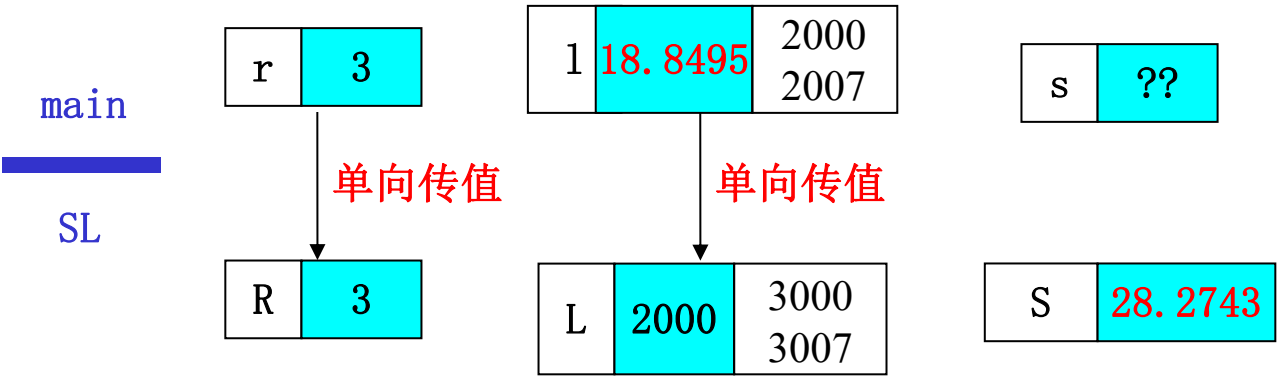
初始内存分配如图所示
请自行画出SL中三句话
执行时内存的变化
理解最后的输出结果



语句: $S = PI * R * R;$



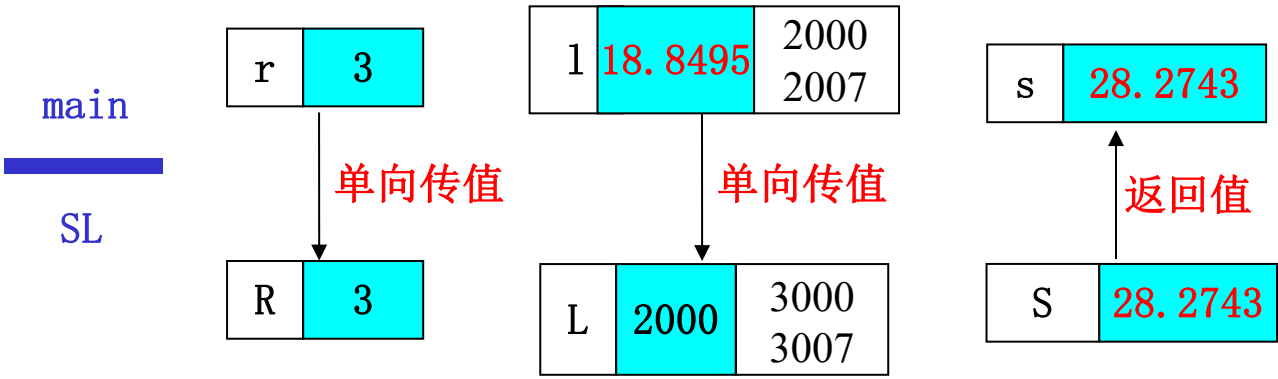
语句: $*L = 2 * PI * R;$



分析:

周长的改变: 通过将半径 **r** 的值传入 **SL** 函数变量 **R**, 计算出周长, 通过函数 **SL** 中的指针变量形参 **L** 访问变量 **I** 的地址并修改其值为周长。

语句: `return S;`



面积的改变: 通过将半径 **r** 的值传入 **SL** 函数变量 **R**, 计算出面积并赋值给变量 **S**, 将函数 **SL** 的返回值 **S** 赋值给变量 **s** 得到面积。



§ 6. 指针基础 – 画内存图并分析

★ PDF课件的P. 32（为什么无法进行交换）

```
void swap(int *x, int *y)
{
    int *t;
    t = x;
    x = y;
    y = t;
}

int main()
{
    int i=10, j=15;
    cout << "i=" << i << " j=" << j << endl;
    swap(&i, &j);
    cout << "i=" << i << " j=" << j << endl;
}
```

初始内存分配如图所示
请自行画出swap中三句话
执行时内存的变化
理解为什么无法交换

单向传值

i	10	2000 2003
---	----	--------------

j	15	2100 2103
---	----	--------------

x	2000	3000 3003
---	------	--------------

y	2100	3100 3103
---	------	--------------

t	?	4000 4003
---	---	--------------

~~i=10 j=15~~ ~~i=10 j=15~~



语句: $t = x;$

i	10	2000 2003
---	----	--------------

单向传值

x	2000	3000 3003
---	------	--------------

j	15	2100 2103
---	----	--------------

单向传值

y	2100	3100 3103
---	------	--------------

t	2000	4000 4003
---	------	--------------

语句: $x = y;$

i	10	2000 2003
---	----	--------------

单向传值

x	2100	3000 3003
---	------	--------------

j	15	2100 2103
---	----	--------------

单向传值

y	2100	3100 3103
---	------	--------------

t	2000	4000 4003
---	------	--------------

分析:

虽然x和y是指向i和j的指针变量，但是本题并没有通过指针访问i和j的地址，而是直接交换x和y的值，这实际上是交换了x和y指向的地址，而对i和j的值没有影响，因此没有交换i和j的值。

语句: $y = t;$

i	10	2000 2003
---	----	--------------

单向传值

x	2100	3000 3003
---	------	--------------

j	15	2100 2103
---	----	--------------

单向传值

y	2000	3100 3103
---	------	--------------

t	2000	4000 4003
---	------	--------------



§ 6. 指针基础 – 画内存图并分析

★ PDF课件的P. 33（为什么会出现错误，导致错误的关键词是哪一句）

```
void swap(int *x, int *y)
{
    int *t;
    *t = *x;
    *x = *y;
    *y = *t;
}

int main()
{
    int i=10, j=15;
    cout << "i=" << i << " j=" << j << endl;
    swap(&i, &j);
    cout << "i=" << i << " j=" << j << endl;
}
```

VS编译报错
-使用了未初始化的局部变量t
其它编译器可能可以运行
初始内存分配如图所示，请自行画出swap中三句话执行时内存的变化，理解为什么出现严重错误
另1：哪句是错误的键？
另2：int *t 改为 int tt, *t;
t = &tt;
为什么就正确了？

i	10	2000
		2003

j	15	2100
		2103

单向传值

x	2000	3000
		3003

y	2100	3100
		3103

t	(假设5000)	4000
		4003

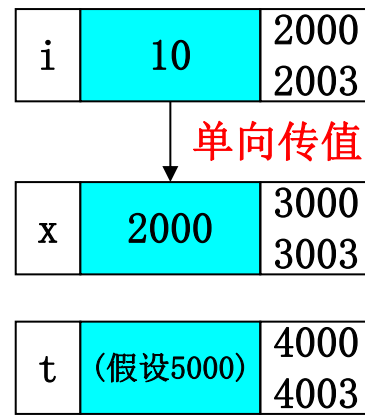
	?	5000
		5003

提示：5000-5003系统是否分配给了程序？

i=10 j=15
i=15 j=10
或 死机或其它非正常现象



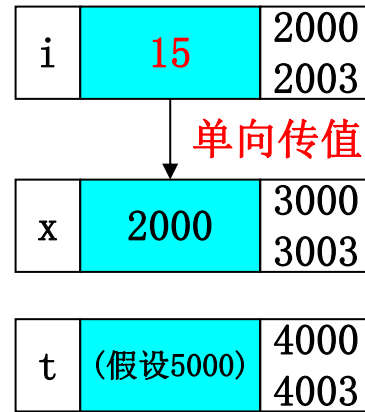
语句: `*t = *x;`



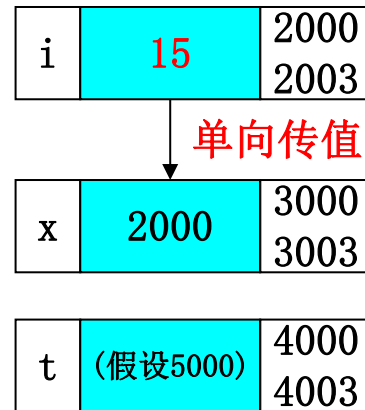
分析:

本程序中, `swap`函数中直接定义了一个指正变量`t`, 而并没有对`t`初始化, 这时`t`指向的地址是随机的(假设为5000), 然后语句`*t = *x;`将`x`指向地址中的`i`的值10存入`t`指向的地址5000-5003, 再进行之后的交换。这样从局部看是可以成功交换`i`和`j`的值, 但是注意地址5000-5003并没有被系统分配给本程序, 本程序是没有对其的访问权限的。本程序中的操作非法修改权限之外的内存, 可能会出现意料之外的错误(如死机蓝屏等), 也有可能不出现错误, 取决于5000-5003地址中存的数据。

语句: `*x = *y;`



语句: `*y = *t;`



错误的关键是`int *t;`这一句`t`指向随机地址, 导致之后的访问和修改非法地址中的数据。改为 `int tt, *t; t = &tt;`之后, `t`指向的地址为变量`tt`的地址, 变量`tt`被定义后, 其地址已经被系统分给程序, 因此不会出现非法访问的情况, 也就不出错。